

06. 차원 축소

✓ 01: 차원 축소(Dimension Reduction) 개요

🔧 차원 축소란?

- 정의: 고차원 데이터를 보다 적은 수의 차원으로 변환하여, 본질적인 특성을 유지하면서 데이터 구조를 간결하게 만드는 과정
- 목적:
 - 데이터 압축
 - 시각화 가능 (2D, 3D)
 - 과적합 방지
 - 계산 효율 향상
 - 데이터의 잠재적 특성(latent feature) 추출

🔧 차원 축소의 필요성

문제 상황	설명
고차원 희소성	차원이 높아질수록 포인트 간의 거리 증가 → 데이터 희소성(Sparsity) 증가
다중 공선성 문제	피처 간 상관관계가 높으면 선형 모델의 예측 성능 저하
시각화 어려움	3차원 이상은 사람의 시각으로 분석이 불가능
과적합	고차원일수록 모델이 훈련 데이터에 과하게 적합됨
학습 시간 증가	계산량과 메모리 사용량 증가

🔧 차원 축소의 구분

- Feature Selection (피처 선택)
 - 불필요한 특성을 제거하고 중요한 것만 선택
- Feature Extraction (피처 추출)
 - 기존 피처를 조합하여 새로운 함축적 피처를 생성

🔧 예시 : 학생 평가 데이터

- 원래 피처들:

- 모의고사, 수능, 내신, 봉사활동, 수상경력 등
- 축소 후 잠재 요인(latent factor):
 - 학업 성취도, 문제 해결력, 커뮤니케이션 능력

차원 축소의 핵심

- 단순히 차원을 줄이는 게 아니라, 기존 피쳐로는 드러나지 않던 구조나 의미를 새롭게 드러내는 것
- 예: 문서 내 주제(topic), 이미지 내 공통 구조

활용 분야

- 이미지 분석
 - 픽셀 수가 매우 많음 → 차원 축소로 정보 압축, 과적합 방지
- 텍스트 분석
 - 단어의 조합에서 의미·주제(semantic topic) 추출

주요 차원 축소 알고리즘

- PCA
- LDA
- SVD
- NMF

02: PCA (Principal Component Analysis)

PCA란?

- 가장 널리 쓰이는 차원 축소(dimensionality reduction) 기법
- 변수들 간의 상관관계를 분석해, 데이터의 변동성을 가장 잘 설명하는 축(주성분)을 찾는다.
- 목표:
 - 정보 손실을 최소화하면서 데이터를 더 적은 수의 축(차원)으로 표현하는 것

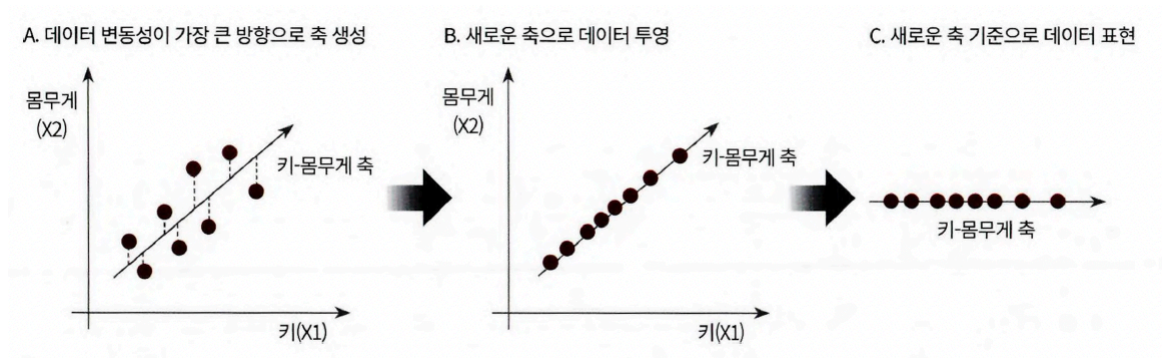
PCA 사용 이유

- 피쳐(변수)가 너무 많으면:

- 희소성(sparsity) 증가 → 학습 어려움
- 공선성(multicollinearity) 발생 → 성능 저하
- 시각화가 어려움 (3D 초과)
- PCA는 데이터를 요약하고 중요한 패턴만 남겨준다.

🔧 핵심 원리

- PCA는 분산이 가장 큰 방향을 찾습니다.
(데이터가 가장 넓게 퍼져 있는 축 → 가장 중요한 정보가 담긴 축)
- 새로운 축들은 서로 직각(orthogonal)
- 이렇게 만들어진 새 축(주성분) 위에 데이터를 투영해서 차원을 줄인다



🔧 용어들

용어	뜻
분산(Variance)	하나의 변수 값들이 평균에서 얼마나 퍼져 있는지
공분산(Covariance)	두 변수(X, Y)가 같이 증가/감소하는 정도
공분산 행렬	여러 변수 간 공분산을 모아 놓은 대칭 정방행렬
고유벡터	행렬에 곱해도 방향은 변하지 않고 크기만 변하는 벡터
고유값	고유벡터가 얼마나 늘어나거나 줄어드는지 나타내는 값

🔧 공분산 예시

	X	Y	Z
X	3.0	-0.71	-0.24
Y	-0.71	4.5	0.28
Z	-0.24	0.28	0.91

- 대각선 원소: 각 변수(X, Y, Z)의 분산
 - X, Y, Z의 분산 → 각각 3.0, 4.5, 0.91
- 대각선 이외의 원소: 가능한 모든 변수의 공분산
 - X과 Y의 공분산 → -0.71
 - X과 Z의 공분산 → -0.24
 - Y과 Z의 공분산 → 0.28

선형대수로 본 PCA

1. 원본 데이터로 공분산 행렬 생성
2. 공분산 행렬로 고유값 분해
 - 고유벡터 = 주성분 방향
 - 고유값 = 각 방향의 분산 크기
3. 고유값이 큰 순서대로 고유벡터 선택 → 데이터 변환

$$C = P \Sigma P^T$$

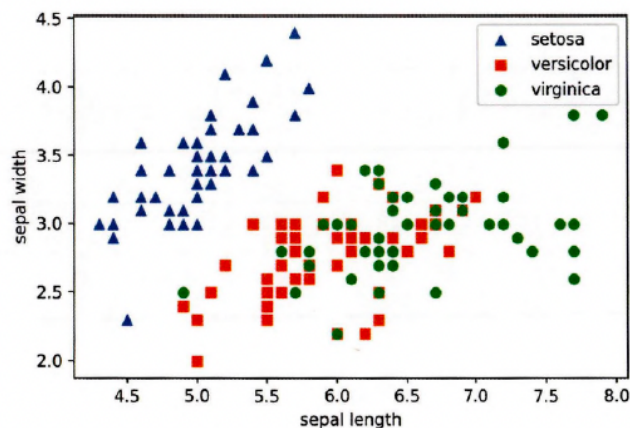
$$C = [e_1 \cdots e_n] \begin{bmatrix} \lambda_1 & \cdots & 0 \\ \cdots & \cdots & \cdots \\ 0 & \cdots & \lambda_n \end{bmatrix} \begin{bmatrix} e_1^t \\ \cdots \\ e_n^t \end{bmatrix}$$

- P: 고유벡터(직교행렬)

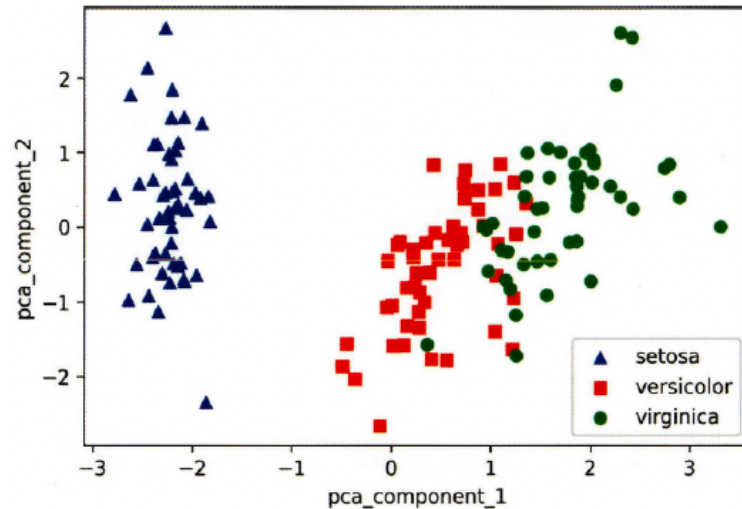
- λ : 고유값(대각행렬)
- P^T : 고유벡터의 전치행렬

🔧 Iris 데이터셋을 활용한 PCA 실습

- 데이터 준비 및 시각화
 - Iris 데이터셋: `sepal_length`, `sepal_width`, `petal_length`, `petal_width` 총 4개 피쳐
 - 2개의 피쳐(`sepal_length`, `sepal_width`)만 사용해 2D 시각화 수행
 - Setosa는 세모, Versicolor는 네모, Virginica는 동그라미로 표현
 - 결과 :



- Setosa는 잘 구분되지만, Versicolor와 Virginica는 겹치는 구간이 있어 단순 시각화로는 어려움
- PCA 적용 전 전처리 (스케일링)
 - StandardScaler 적용
- PCA 적용 (4D → 2D)
 - `PCA(n_components=2)` (2개의 주성분 추출)
 - `fit()` → `transform()` 수행 → 150×2 차원의 데이터로 변환
- 새로운 컬럼들:
 - `pca_component_1`, `pca_component_2`
- PCA 결과 시각화



- setosa, versicolor, virginica의 분포가 더 명확하게 표현됨, 특히 pca_component_1이 setosa를 잘 분리해냄
- 주성분들의 중요도 확인
 - explained_variance_ratio → [0.72962445 0.22850762]
 - 1번 축이 전체 분산의 72.9%, 2번 축이 22.8% 설명 → 2개 축만으로도 95.8% 정보 보존
- 예측 성능 비교 (Random Forest)
 - 원본 (4D)
 - 평균 정확도 : 0.96
 - PCA 변환 (2D)
 - 평균 정확도 : 0.88
 - 정확도 하락 약 8%, 하지만 피쳐 50% 감소한것 기 때문에 원본 데이터 특성 유지하고 있음

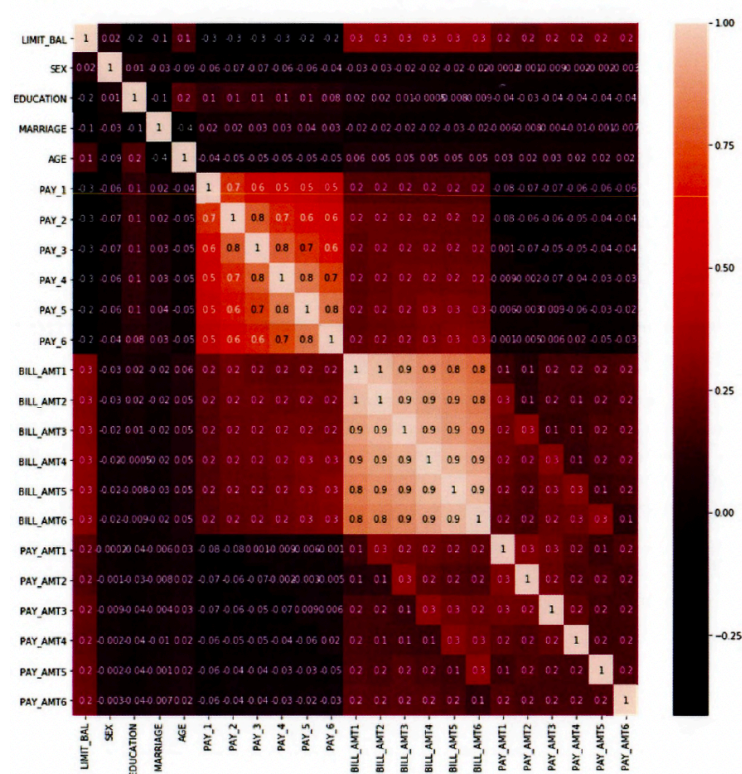
🔧 신용카드 고객 데이터에 대한 PCA 적용 개념 정리

- 데이터세트
 - 출처: UCI Machine Learning Repository – Credit Card Clients Dataset
 - 30,000개의 고객 데이터, 24개의 특성
 - 목표 변수(Target): default payment next month [다음 달 연체 여부 (0: 정상, 1: 연체)]
- 전처리

- 컬럼명 변경

- `PAY_0` → `PAY_1`
- `default payment next month` → `default`

- 상관계수 분석



- `BILL_AMT1` to `BILL_AMT6`: 서로 상관계수 0.9 이상
- `PAY_1` to `PAY_6`: 상관계수 높음
- 이런 경우 PCA로 소수의 축으로 압축 가능성이 높음!

- PCA 수행 (BILL_AMT만 추출)

- `BILL_AMT1` to `BILL_AMT6` (총 6개 변수)
- StandardScaler
- `n_components=2`로 설정 후 PCA 수행
- 결과:

- `explained_variance_ratio_` [0.9055, 0.0510]

- 첫 번째 축에서 90% 이상 정보 유지, 두 축만으로도 전체 변동성 95% 이상 설명 가능

- 분류 성능 비교 (Random Forest + 교차검증)
 - 원본 23개 피쳐 사용:
 - 평균 정확도: 0.8170
 - PCA : 6개 컴포넌트로 축소:
 - 평균 정확도: 0.7973
 - 단 6개의 특성만으로도 약 81.7% → 79.7%로 2% 미만의 성능 감소 → 높은 압축률 대비 성능 유지력 우수

✓ 03: LDA (Linear Discriminant Analysis)

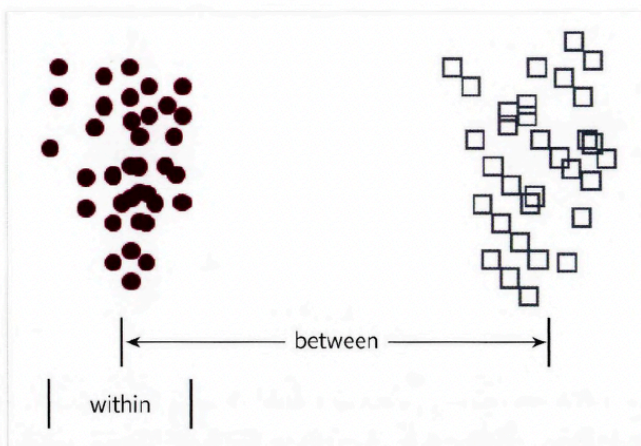
🔧 LDA란?

- 선형 판별 분석법으로, PCA와 유사하게 차원 축소를 수행하는 기법
- 분류(classification)를 목적으로 한 지도학습 기반의 차원 축소

🔧 PCA vs. LDA 차이

	PCA	LDA
목적	최대 분산 방향 찾기	클래스 분리가 잘 되도록 축 찾기
학습 방식	비지도학습	지도학습 (클래스 라벨 필요)
수학적 기반	공분산 행렬	클래스 간/내 분산 행렬
주 용도	데이터 압축/시각화	분류 성능 향상

🔧 LDA의 핵심 개념



LDA는 클래스 분리를 극대화하는 새로운 축을 찾습니다.

- 클래스 간 분산(between-class scatter):
 - 클래스 평균 사이의 거리. 크게 만들수록 좋음.
- 클래스 내 분산(within-class scatter):
 - 같은 클래스 내 데이터의 퍼짐 정도. 작게 만들수록 좋음.
- 클래스 간은 멀리, 클래스 내는 가까이 함으로 분류 성능 향상

LDA 수행 단계

1. 클래스 내/클래스 간 분산 행렬 계산
 - 각 클래스의 평균 벡터 사용
2. 두 행렬 기반으로 고유벡터 분해
 - 비율을 최대화하는 축 도출
3. 고유값이 큰 순으로 K개의 고유벡터 선택
4. 선택된 고유벡터로 데이터 투영 → 차원 축소

LDA 수식

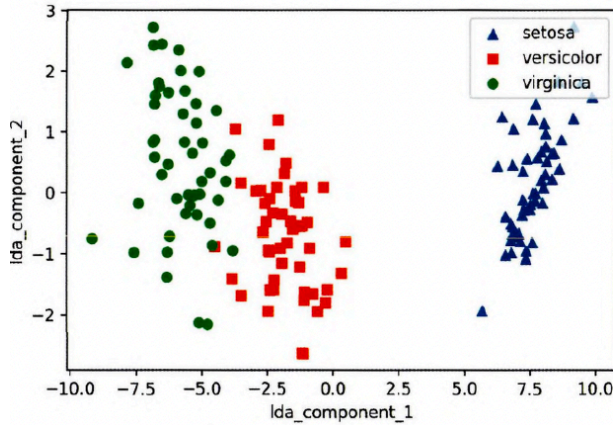
$$S_W^T S_B = \begin{bmatrix} e_1 & \cdots & e_n \end{bmatrix} \begin{bmatrix} \lambda_1 & \cdots & 0 \\ \cdots & \cdots & \cdots \\ 0 & \cdots & \lambda_n \end{bmatrix} \begin{bmatrix} e_1^T \\ \cdots \\ e_n^T \end{bmatrix}$$

붓꽃 데이터에 LDA 적용

- 목표: LDA로 4D → 2D 축소
- StandardScaler()이용하고 전처리
- LDA 변환
 - `lda = LinearDiscriminantAnalysis(n_components=2)`
 - 주의: LDA는 `fit()` 시 `target` 필요

- 시각화

3) 시각화



- PCA처럼 2D로 표현됨
- 클래스 간 경계가 더 명확함

✓ 04: SVD (Singular Value Decomposition)

🔧 SVD란?

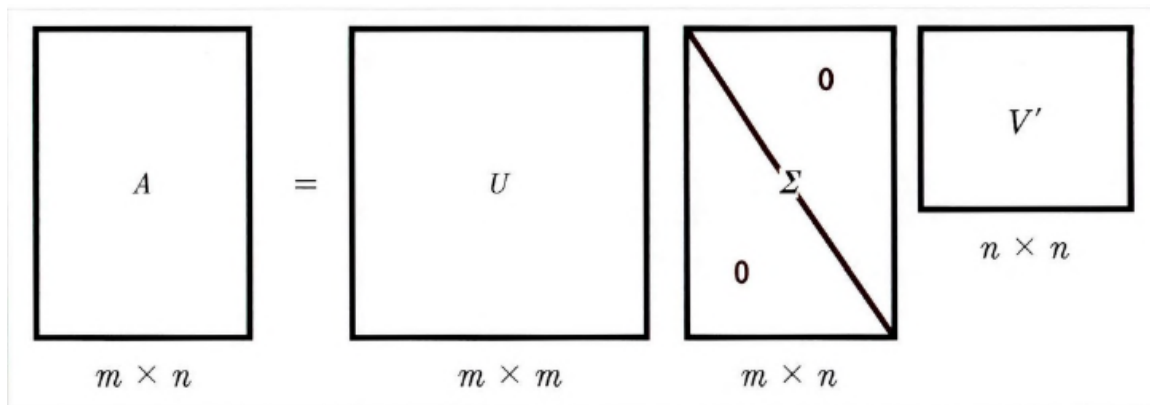
- 행렬을 세 개의 특수한 행렬로 분해하는 행렬 분해 기법
- 행과 열의 개수가 같지 않아도 적용 가능하고, PCA보다 더 일반적인 형태의 차원 축소 도구

🔧 SVD수식

$$A = U \Sigma V^T$$

기호	설명
U	특이벡터 (orthogonal matrix)
Σ \Sigma	대각 행렬, 특이값(singular values)을 가짐
V^T	특이벡터의 전치 행렬

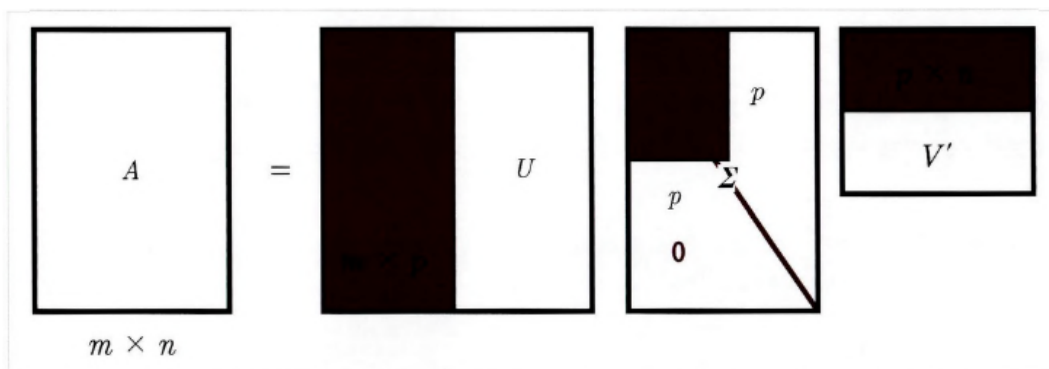
🔧 일반적인 SVD 차원



- A : $m \times n$ times
- U : $m \times m$ times
- Σ : $m \times n$ times (대각선만 값 존재)
- V^T : $n \times n$ times

🔧 Truncated SVD (축소 SVD)

- 특이값 상위 일부만 유지하고 나머지는 제거해 차원 축소
- 행렬의 차원이 줄어들고 계산 효율성 증가



🔧 실험 요약 (예시 실습)

🔧 일반 SVD

- 넘파이의 `numpy.linalg.svd()` 사용
- 임의의 $4 \times 4 \times 4$ 행렬을 U, Σ, V^T 로 분해

- 로우 간 의존성이 클 경우 → Sigma에 0 등장 → 차원 축소 가능성 증가

🔧 Truncated SVD

- `scipy.sparse.linalg.svds()` 또는 `sklearn.TruncatedSVD` 사용
- 일부 상위 특이값만 사용하여 차원 축소
- 복원은 근사적으로만 가능하지만 성능 유지 가능

🔧 사이킷런 TruncatedSVD

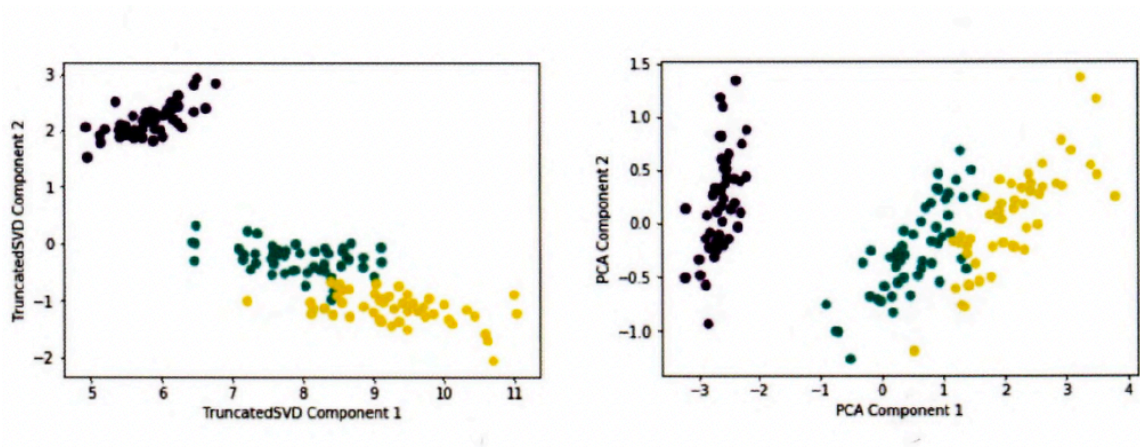
```
from sklearn.decomposition import TruncatedSVD
svd = TruncatedSVD(n_components=2)
X_reduced = svd.fit_transform(data)
```

- `fit()` 과 `transform()` 을 PCA처럼 사용
- PCA와 매우 유사한 결과 → 특히 스케일링 후엔 결과 거의 동일

🔧 활용

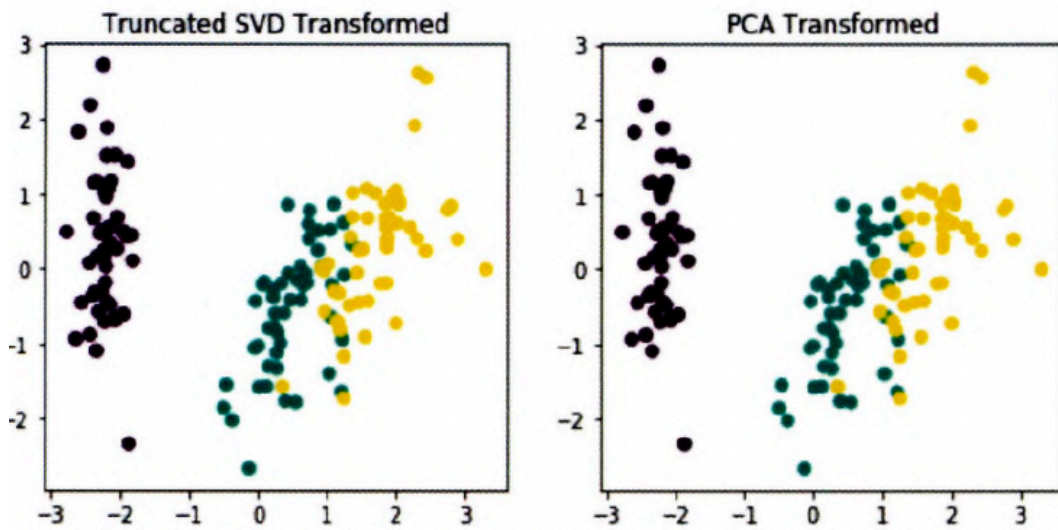
- 이미지 압축 / 신호 처리
- 텍스트 분석 (LSA)
 - 문서-단어 행렬에서 의미 요약
- 추천 시스템에서 잠재 요인 추출

🔧 PCA vs. SVD



- 유사한 결과: 어느 정도 클러스터링이 가능한 정도

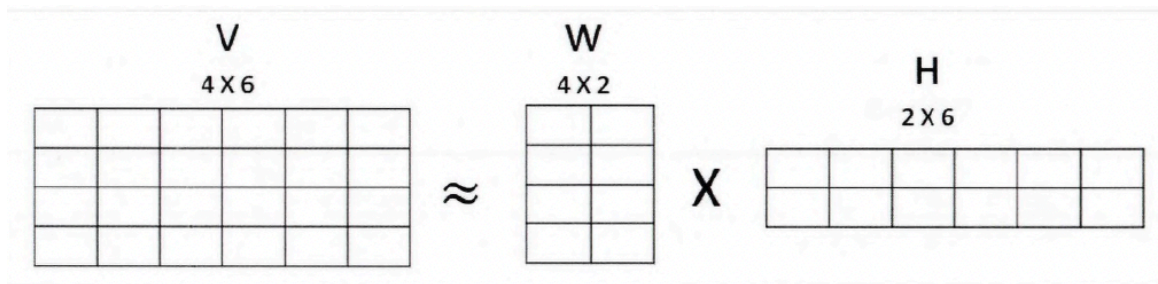
- ScikitLearn TruncatedSVD와 PCA 클래스 두개 모두 SVD를 이용해 행렬을 분해 하기 때문에 스케이링 후 거의 동일함을 알 수 있음



✓ 05: NMF (Non-Negative Matrix Factorization)

🔧 NMF란?

- 모든 행렬 요소가 0 이상(양수)일 때만 사용할 수 있는 행렬 분해 기법
- 낮은 차원으로 행렬을 분해하고 데이터 압축, 잠재 요인 추출, 차원 축소 등에 활용됨



기호	설명
V	원본 행렬 ($m \times n$)
W	행 기반 잠재 요소 행렬 ($m \times k$)
H	열 기반 잠재 요소 행렬 ($k \times n$)

- W: 각 데이터 포인트(행)가 잠재 요인을 얼마나 가지는지
- H: 각 특성(열)이 잠재 요인과 얼마나 관련 있는지

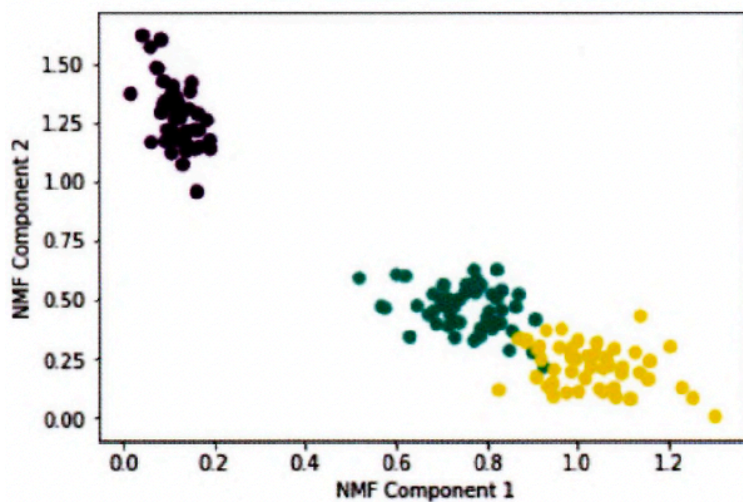
- 이 두 행렬을 곱하면, 원본 데이터와 거의 유사한 값을 근사할 수 있음

🔧 붓꽃 데이터에 NMF 적용

- 붓꽃 데이터는 전처리 후 NMF로 2개 컴포넌트로 축소
- `sklearn.decomposition.NMF` 클래스로 수행

```
from sklearn.decomposition import NMF
```

```
nmf = NMF(n_components=2)
iris_nmf = nmf.fit_transform(iris_data)
```



- 시각화 결과, 클래스별 분포 차이가 어느 정도 드러남

🔧 활용

- 텍스트 분석
- 추천 시스템
 - 사용자-상품 평점 행렬 → 사용자/상품 잠재 요인 추출
 - 이런 방식을 잠재 요인 모델(Latent Factoring Model) 이라고 부름
- 이미지 처리
- 잠재 요인을 기반으로 유사도/군집 분석

물론입니다! 아래는 차원 축소(Dimensionality Reduction) 챕터 전체 요약입니다. PCA, LDA, SVD, NMF 네 가지 주요 알고리즘의 핵심 목적과 차이점을 중심으로 정리했습니다.

✓ 06: 정리

🔧 차원 축소란?

- 많은 피처(속성)를 가진 고차원 데이터를 더 작고 의미 있는 축소된 차원으로 표현하는 방법
- 단순히 피처 수를 줄이는 것을 넘어서 잠재적인 의미(요인, 패턴)를 추출 가능
- 이미지, 텍스트, 추천 시스템 등에서 자주 활용됨

🔧 대표 알고리즘 요약

알고리즘	핵심 아이디어	목적 / 특성
PCA (주성분 분석)	분산(변동성)이 가장 큰 축을 찾아 데이터를 투영	비지도학습 / 압축 / 시각화 / 선형 변환
LDA (선형 판별 분석)	클래스 간 분리(클래스 간 분산 up, 클래스 내 분산 down)	지도학습 / 분류 성능 향상 / 클래스 분리
SVD (특이값 분해)	고차원 행렬을 3개의 직교 행렬로 분해	일반적인 행렬 분해 / Truncated SVD
NMF (비 음수 행렬 분해)	양수만 포함된 행렬을 두 개의 양수 행렬로 분해	해석 가능성 up / 추천 시스템

🔧 활용

- PCA: 이미지 압축, 시각화, 사전 차원 축소
- LDA: 텍스트 분류, 이미지 분류 등 라벨 정보 기반 분류 성능 개선
- SVD: LSA(Latent Semantic Analysis)
- NMF: 주제 분류, 추천 시스템 등